

Java8并行流有坑！慎用

在说 `ParallelStream` 之前，一定要了解 `Stream` 以及它的基本操作。

什么是 `ParallelStream`

上文讲到的 Java8 `Stream` 流在执行时候是串行化的，如果说任务执行的耗时比较长，可以使用 `Stream` 的 "兄弟流" `ParallelStream`。

防止误导，并非耗时就一定要使用并行，根据不同的业务场景，合理的使用即可。

`parallelStream` 是一种并行流，意思为处理任务时并行处理，这里和并发编程就有了千丝万缕的关系。

前提是硬件支持，如果单核 CPU，只会存在并发处理，而不会并行。

这篇文章主要是说明 `ParallelStream` 其中一个可能成为埋雷的点。

那项目中业务使用的并行流真的会都并行处理么？

如何使用 `ParallelStream`

`ParallelStream` 在使用上与 `Stream` 无区别，本质返回的都是一个流，只不过底层处理时 **根据条件判断是并行或者是串行**。

```
3 import com.google.common.collect.Lists;
4
5 /**
6  * 测试并行流
7  *
8  * @author macher
9  * @date 2020/5/16 21:57
10 */
11 public class ParallelStreamTest {
12
13     public static void main(String[] args) {
14         Lists.newArrayList("1", "2", "3", "4")
15             .parallelStream()
16             .forEach(System.out::println);
17     }
18 }
19
20 ParallelStreamTest - main()
21
22 Debug: ParallelStreamTest
23
24 Debugger Console
```

并行流并不会按照原本的顺序轨迹执行，而是 \随机执行，\当然对于这种 forEach 输出也可以做到顺序串行，但这个不在文章中的重点。

ForkJoinPool

相信如果在项目中实际使用过并行流的小伙伴，一定会知道 ForkJoinPool。没错，并行流底层就是使用的 ForkJoinPool，一种 **工作窃取算法线程池**。

ForkJoinPool 的优势在于，**可以充分利用多 CPU 的优势**，把一个任务拆分成多个"小任务"，把多个"小任务"放到多个处理器核心上并行执行；当多个"小任务"执行完成之后，再将这些执行结果合并起来。

并行流的陷阱

1. 线程安全问题

只要是并行处理，如果在流程中的操作产生了竞态条件，就会存在线程安全问题。

这里举个例子进行说明具体问题。

```

public static void main(String[] args) {
    List<Integer> integerList = Lists.newArrayList();
    List<String> strList = Lists.newArrayList();

    int practicalSize = 1000000;

    for (int i = 0; i < practicalSize; i++) {
        strList.add(String.valueOf(i));
    }

    strList.parallelStream().forEach(each -> {
        integerList.add(Integer.parseInt(each));
    });

    log.info(" >>> integerList 预计长度 :: {}", practicalSize);
    log.info(" >>> integerList 实际长度 :: {}", integerList.size());
}
/**
 * >>> integerList 预估长度 :: 1000000
 * >>> integerList 实际长度 :: 211195
 */

```

上面这段程序运行流程说明如下:

1. 创建了两个 List，分别是 String、Integer 类型。
2. 向 strList 插入 1000000 条记录。
3. 使用并行流将 strList 中的数据格式化为 Integer 并添加到 integerList。
4. 输出 integerList 预计长度以及实际长度。

正常情况下，我们是希望 integerList 最终输出 1000000。但是会因为并行流处理是多线程操作，所以会导致 ArrayList 的线程不安全。

示例中实际长度并不固定，根据 CPU 的具体处理速度而定。

2. 线程安全解决方式

如果项目中确实有上述代码的需求，可以选择使用 **Vector 类**、**Colletions 封装**、**JUC 类**。

既然使用了并行处理，所以对于性能还是有一定要求，所以这一块容器偏向于 JUC。

3. 什么情况下都会并行么

首先我们先将能调用并行流的 API 进行罗列。

```
public static void main(String[] args) {  
    List<String> stringList = Lists.newArrayList();  
    stringList.parallelStream();  
    stringList.stream().parallel();  
    Stream.of(stringList).parallel();  
    ...  
}
```

虽然 API 的调用方式不同，但是底层都是将 **AbstractPipeline** 中的 **parallel** 标识设置为 **true**。

```
public final S parallel() {  
    sourceStage.parallel = true;  
    return (S) this;  
}
```

这就会引出一个问题，调用这三种不同的并行流 API，**底层是使用的同一个 ForkJoinPool 么？**

首先我们看一下 ForkJoinPool 是如何被初始化的，并行流中使用到的是 ForkJoinPool 内部一个静态变量属性。

```
static final ForkJoinPool common;  
  
public static ForkJoinPool commonPool() {  
    // assert common != null : "static init error";  
    return common;  
}
```

ForkJoinPool 静态块负责初始化 common。

```

static {
    // initialize field offsets for CAS etc
    try {
        U = sun.misc.Unsafe.getUnsafe();
        Class<?> k = ForkJoinPool.class;
        CTL = U.objectFieldOffset
            (k.getDeclaredField("ctl"));
        RUNSTATE = U.objectFieldOffset
            (k.getDeclaredField("runState"));
        STEALCOUNTER = U.objectFieldOffset
            (k.getDeclaredField("stealCounter"));
        Class<?> tk = Thread.class;
        PARKBLOCKER = U.objectFieldOffset
            (tk.getDeclaredField("parkBlocker"));
        Class<?> wk = WorkQueue.class;
        QTOP = U.objectFieldOffset
            (wk.getDeclaredField("top"));
        QLOCK = U.objectFieldOffset
            (wk.getDeclaredField("qlock"));
        QSCANSTATE = U.objectFieldOffset
            (wk.getDeclaredField("scanState"));
        QPARKER = U.objectFieldOffset
            (wk.getDeclaredField("parker"));
        QCURRENTSTEAL = U.objectFieldOffset
            (wk.getDeclaredField("currentSteal"));
        QCURRENTJOIN = U.objectFieldOffset
            (wk.getDeclaredField("currentJoin"));
        Class<?> ak = ForkJoinTask[].class;
        ABASE = U.arrayBaseOffset(ak);
        int scale = U.arrayIndexScale(ak);
        if ((scale & (scale - 1)) != 0)
            throw new Error("data type scale not a power of two");
        ASHIFT = 31 - Integer.numberOfLeadingZeros(scale);
    } catch (Exception e) {
        throw new Error(e);
    }

    commonMaxSpares = DEFAULT_COMMON_MAX_SPARES;
    defaultForkJoinWorkerThreadFactory =
        new ForkJoinPool.DefaultForkJoinWorkerThreadFactory();
    modifyThreadPermission = new RuntimePermission("modifyThread");
}

```

```

        // 创建ForkJoinPool
        common = java.security.AccessController.doPrivileged
            (new java.security.PrivilegedAction<ForkJoinPool>() {
                public ForkJoinPool run() {
                    return makeCommonPool();
                }
            });
        int par = common.config & SMASK; // report 1 even if threads disabled
        commonParallelism = par > 0 ? par : 1;
    }

```

通过下面初始化代码可以看到，parallelism、threadFactory、exceptionHandler 可以进行初始个性化配置。

```

private static ForkJoinPool makeCommonPool() {
    int parallelism = -1;
    ForkJoinPool.ForkJoinWorkerThreadFactory factory = null;
    Thread.UncaughtExceptionHandler handler = null;
    try { // ignore exceptions in accessing/parsing properties
        String pp = System.getProperty
            ("java.util.concurrent.ForkJoinPool.common.parallelism");
        String fp = System.getProperty
            ("java.util.concurrent.ForkJoinPool.common.threadFactory");
        String hp = System.getProperty
            ("java.util.concurrent.ForkJoinPool.common.exceptionHandler");
        if (pp != null)
            parallelism = Integer.parseInt(pp);
        if (fp != null)
            factory = ((ForkJoinPool.ForkJoinWorkerThreadFactory) ClassLoader.
                getSystemClassLoader().loadClass(fp).newInstance());
        if (hp != null)
            handler = ((Thread.UncaughtExceptionHandler) ClassLoader.
                getSystemClassLoader().loadClass(hp).newInstance());
    } catch (Exception ignore) {
    }
    if (factory == null) {
        if (System.getSecurityManager() == null)
            factory = defaultForkJoinWorkerThreadFactory;
        else // use security-managed default

```

```

        factory = new ForkJoinPool.InnocuousForkJoinWorkerThreadFactory();
    }
    if (parallelism < 0 && // default 1 less than #cores
        (parallelism = Runtime.getRuntime().availableProcessors() - 1) <= 0)
        parallelism = 1;
    if (parallelism > MAX_CAP)
        parallelism = MAX_CAP;
    return new ForkJoinPool(parallelism, factory, handler, LIFO_QUEUE,
        "ForkJoinPool.commonPool-worker-");
}

```

创建 ForkJoinPool 实例内部线程总数 parallelism 默认为: \\当前运行环境的 CPU 核数 - 1。\\这一点很重要, 和下面讲到的并行流工作方式有很大关系。

看到这里小伙伴应该就会明白了。

\\程序中使用的并行流, 使用的都是 ForkJoinPool 中的静态变量 common。\\继续看本节提出的问题, 项目中使用了并行流的代码, 真的能够达到并行么?

先贴一下测试代码, 感兴趣的小伙伴可以本地也试试。

```

public static void main(String[] args) throws InterruptedException {
    System.out.println(String.format(" >>> 电脑 CPU 并行处理线程数 :: %s, 并行流公共线程池内线程数 :: %s",
        Runtime.getRuntime().availableProcessors(),
        ForkJoinPool.commonPool().getParallelism()));
    List<String> stringList = Lists.newArrayList();
    List<String> stringList2 = Lists.newArrayList();
    for (int i = 0; i < 13; i++) stringList.add(String.valueOf(i));
    for (int i = 0; i < 3; i++) stringList2.add(String.valueOf(i));

    new Thread(() -> {
        stringList.parallelStream().forEach(each -> {
            System.out.println(Thread.currentThread().getName() + " 开始执行 :: " +
each);
            try {
                Thread.sleep(6000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });
    })

```

```

    });
    }, "子线程-1").start();

    Thread.sleep(1500);

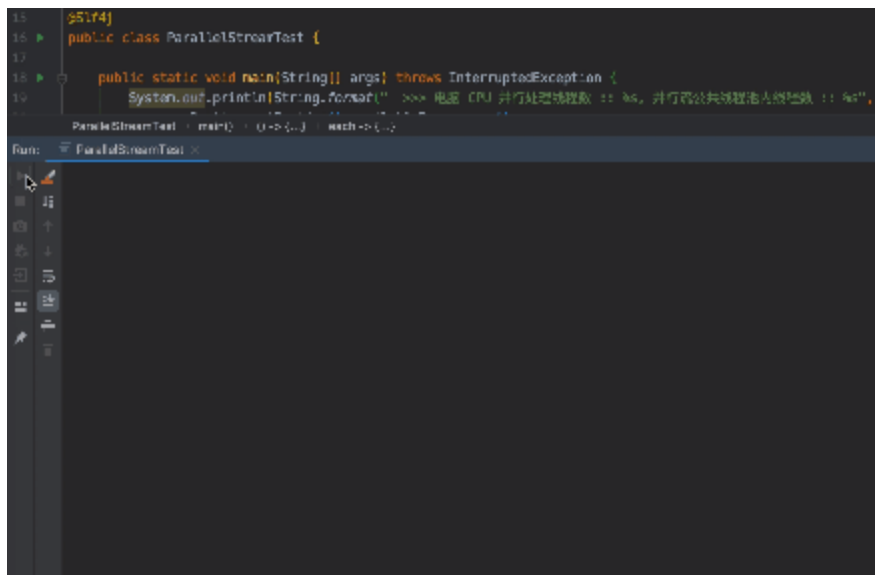
    new Thread(() -> {
        stringList2.parallelStream().forEach(each -> {
            System.out.println(Thread.currentThread().getName() + " :: " + each);
            try {
                Thread.sleep(50);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });
    }, "子线程-2").start();
}

```

为了模拟项目中正式使用场景，测试代码说明如下：

1. "子线程-1"、"子线程-2" 分别代表项目中两个不同的业务使用并行流。
2. 服务器同时只能保证 12 线程并发，初始化时公共的 ForkJoinPool 内并行度是 11。
3. "子线程-1" 业务比较耗时，算上执行线程以及线程池内的线程，并发能跑 12 个任务。
4. 如果 "子线程-1" 将线程池所有并行跑满，"子线程-2" 再运行并行流有什么结果？

跑一下测试程序，看看会发生什么事情。



这里说明下运行图的过程说明:

1. 可以看到提交任务的线程也参与到了任务执行中。
2. 因为我们公共的 ForkJoinPool 并行是 11, 加上提交任务的线程一共是 12, 但是我们 "子线程-1" 共需执行 13 个任务。
3. 在 "子线程-1" 中的任务将线程睡眠, 模拟任务耗时, 所以 "子线程-1" 会将公共线程池跑满的同时, 还会遗留一个任务。
4. 因为 "子线程-1" 将任务跑满, 所以 "子线程-2" 在执行的时候, **不能进行并行处理**, 只能依靠提交任务线程执行。
5. 在 "子线程-1" 的 12 个任务结束运行后, 会再将剩余的一个任务继续执行。

4. 问题总结

通过上面的测试程序得知: 在项目中使用了并行流真正执行时, **并非一定是并行的**。

因为如果项目中其它并行流的任务执行耗时, 会占据对应资源, 使得最后还是通过主线程执行任务。

所以我们在使用并行流之前一定要考虑以下问题:

1. 业务场景是否真的需要并行处理?
2. 并行处理任务是否是相对独立? 是否会引起并行间的竞态条件?
3. 并行处理是否依赖任务的执行顺序?

针对这三个问题, 如果业务能够满足使用场景, 并且有对应的解决策略, 并行确实是能够提升相当一部分性能。

ParallelStream 总结

文章主要描述了什么是 ParallerStream 是什么? 一种提供了并行计算的流式处理。

ParallerStream 底层是通过什么技术获得并行计算? ForkJoinPool, 默认并行能力为 `Runtime.getRuntime().availableProcessors() - 1`, 可以通过参数指定重写。

并行流存在的一些问题, 其实也可以说是并发编程存在的问题。线程安全性问题及场景是否适用并行处理。

总而言之, 并行流处理在合适的场景还是可以使用的。

更新: 2023-08-24 13:29:31

原文: <<https://www.yuque.com/magestack/12306/hgq6dl5r5gfk2np8>>